

Reviewer Pack — Minimal Rank of Quandle Representations over Resolution Proo...

Entry 9b3cecd66db2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 08:47:53 UTC

Minimal Rank of Quandle Representations over Resolution Proof Trees

Entry ID: 9b3cecd66db2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 08:47:53 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For every k -CNF formula F , there exists a quandle Q such that the minimal rank of its representation is equal to the resolution proof complexity of F .’, ‘For all instances F with n variables and m clauses, there exists a quandle Q such that $\text{Rank}(Q) = \alpha(n,m)$, where $\alpha(n,m)$ is the resolution proof complexity of F .’]

Rationale (proposer’s reasoning):

[‘Quandle theory provides a framework for studying algebraic structures that may capture hidden symmetries in computational processes. A quandle representation of a CNF formula could potentially expose non-trivial structure related to its resolution proof complexity.’, ‘This bridge between quandle theory and resolution proof complexity might lead to new insights into the nature of NP-completeness and the limits of efficient computation.’]

Taxonomy category: Quandle_Theory_×_Complexity_Theoretic (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 47e31473c8a4b7af

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all k -CNF formulas F with n variables and m clauses, the computed minimal rank of a quandle Q ’s representation equals the resolution proof complexity $\alpha(n,m)$ with at least 90% of the generated seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n, m):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(m):
            clause = [random.choice(variables) for _ in range(random.randint(1, n))]
            if random.choice([True, False]):
                clause = [-v for v in clause]
            clauses.append(clause)
        return variables, clauses

    def resolution_proof_complexity(variables, clauses):
        # Simplified version of resolution proof complexity calculation
        # This is a placeholder and should be replaced with an actual algorithm
        return len(clauses) * 2

    def quandle_rank(variables, clauses):
        # Placeholder for quandle rank calculation
        # This is a placeholder and should be replaced with an actual algorithm
        return random.randint(1, 50)

    n = random.choice([5, 10, 15, 20, 30, 40])
    m = random.randint(n, 2 * n)
    variables, clauses = generate_k_cnf(n, m)

    alpha_nm = resolution_proof_complexity(variables, clauses)
    rank_q = quandle_rank(variables, clauses)
```

```

return {
    "metric_name": "Quandle Rank",
    "metric_value": rank_q,
    "instances_tested": 1,
    "conjecture_holds": rank_q == alpha_nm,
    "counterexample": f"Mean rank {rank_q} does not match resolution complexity {alpha_nm}" if
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_d = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_d) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_d} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_d} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

resolution complexity 10'}
TRIAL: {'metric_name': 'Quandle Rank', 'metric_value': 44, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'Quandle Rank', 'metric_value': 7, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'Quandle Rank', 'metric_value': 48, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'Quandle Rank', 'metric_value': 23, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'Quandle Rank', 'metric_value': 38, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'Quandle Rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'Quandle Rank', 'metric_value': 33, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'Quandle Rank', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds':

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ec66bf7.py", line 75, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ec66bf7.py", line 75, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge falsified | original: The test results show that for at least one instance, the mean rank of the quandle does not match the resolution complexity, which contradicts the con | next: Investigate further to identify why the computed minimal ranks do not match the resolution complexities and explore alternative methods or assumptions that could lead to a correct relationship between quandle representations and resolution proof complexity.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11158
2	preregistration	ollama_remote	glm4:latest	0	0	5385
3	novelty	ollama_remote	glm4:latest	0	0	4649
4	novelty	ollama_remote	glm4:latest	0	0	4758
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19614
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7031
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8002
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8283
9	judge	ollama_remote	glm4:latest	0	0	8631

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 77510 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/9b3cecd66db2.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9b3cecd66db2.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9b3cecd66db2.tar.gz (if generated)